

Platform Engineering 201

How to get started with it?



Disclaimer

- Everything is my own opinion; feel free to disagree.
- A lot of content: They are touchpoints to show you what you have to think about.
⇒ Sometimes dense slides, but then you can use them as a guide afterward.
- You are always free to reach out.





Imagine

First day

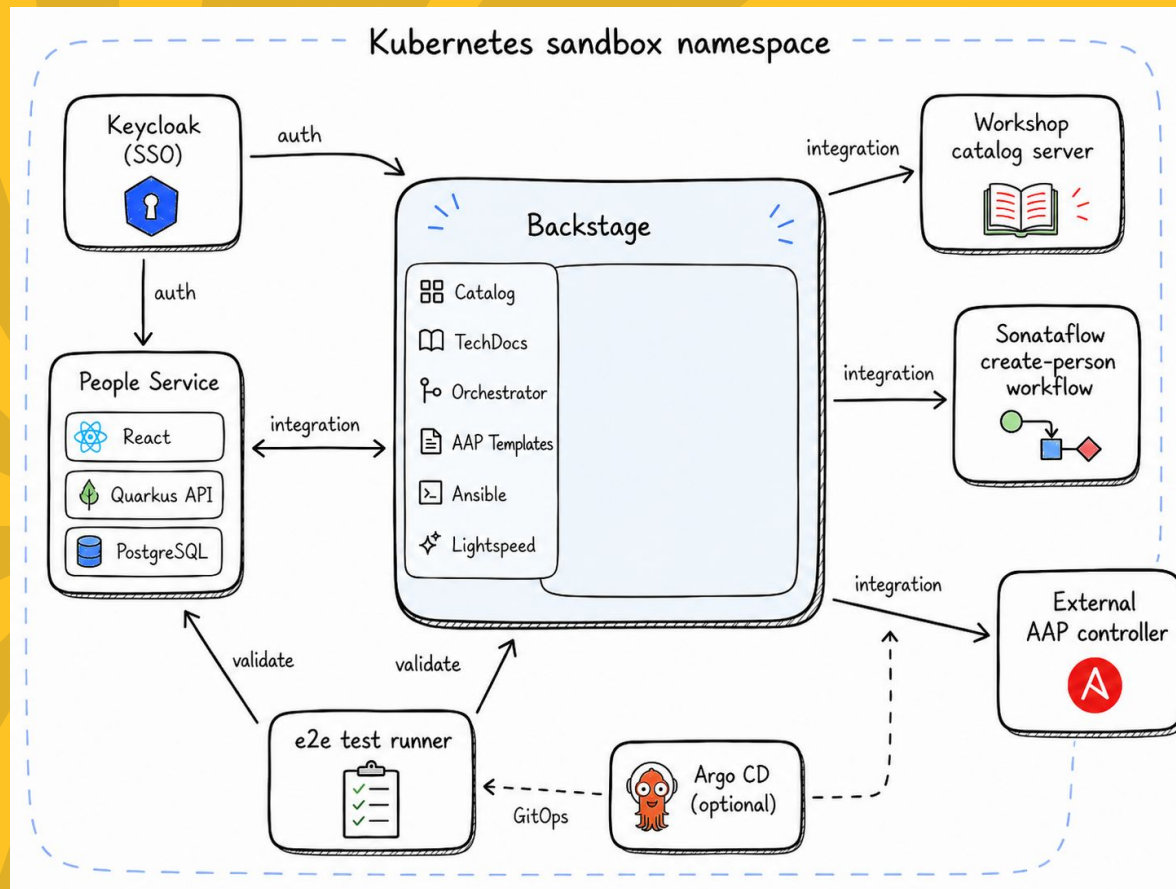
A man with a beard, wearing a red fedora and blue sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. To his right is a large, white, cloud-like thought bubble with a thick black outline. Inside the bubble, the word "Demo" is written in a bold, black, sans-serif font, and "First day" is written in a smaller, black, handwritten-style font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from behind the thought bubble, creating a sense of energy and excitement.

Demo

First day

Agenda





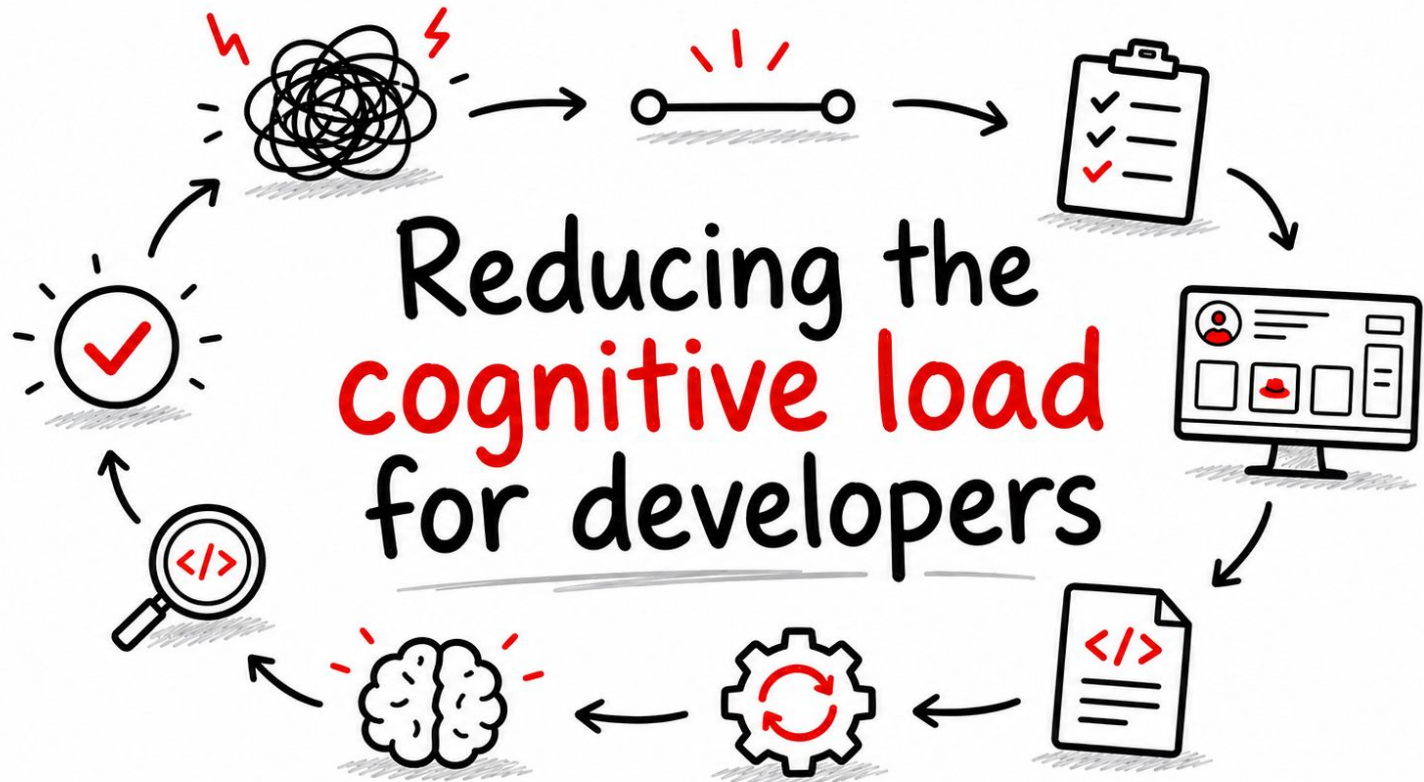
maarten-vandepere.be

<https://github.com/maarten-vandepierre/platform-engineering-201>

Platform engineering

Explaining key concepts

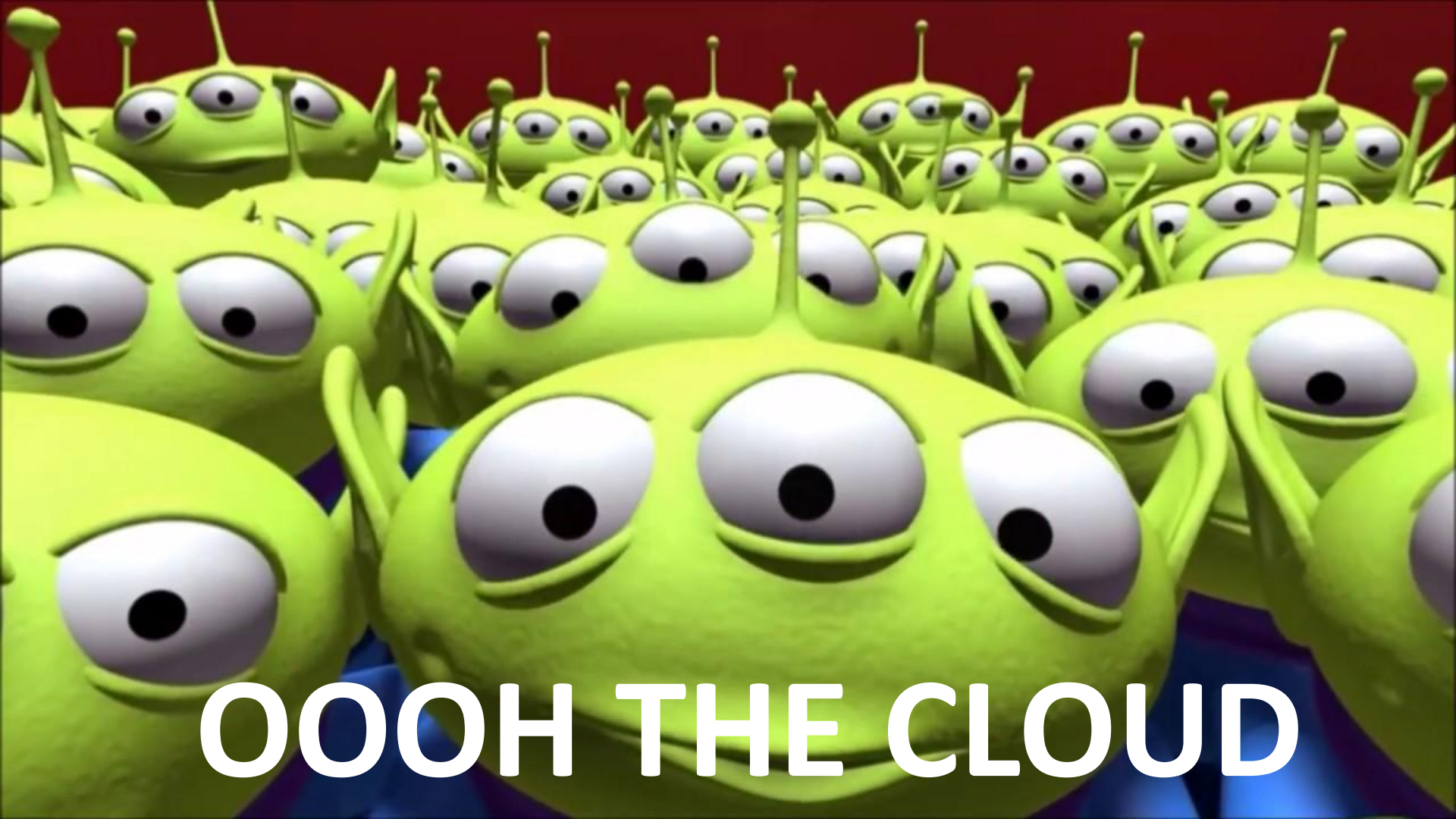






IN THE PAST
DEVELOPERS
WERE **HAPPY**





OOOH THE CLOUD



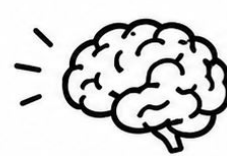
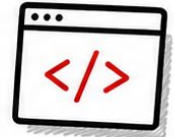
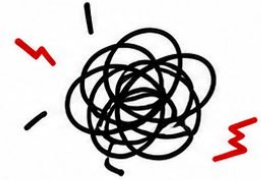
$H_0 = H_1 = H_2 = \dots$
 $H_1 = \sum_{i=1}^n \langle \phi_i | \rho | \phi_i \rangle$
 $H_2 = \sum_{i=1}^n \prod_{j=1}^n \langle \phi_i | \rho | \phi_j \rangle$
 $\dots$
 $H_n = \prod_{i=1}^n \sum_{j=1}^n \langle \phi_i | \rho | \phi_j \rangle$
 $\dots$
 $G_7 = P_7 LS$

YOU BUILD IT

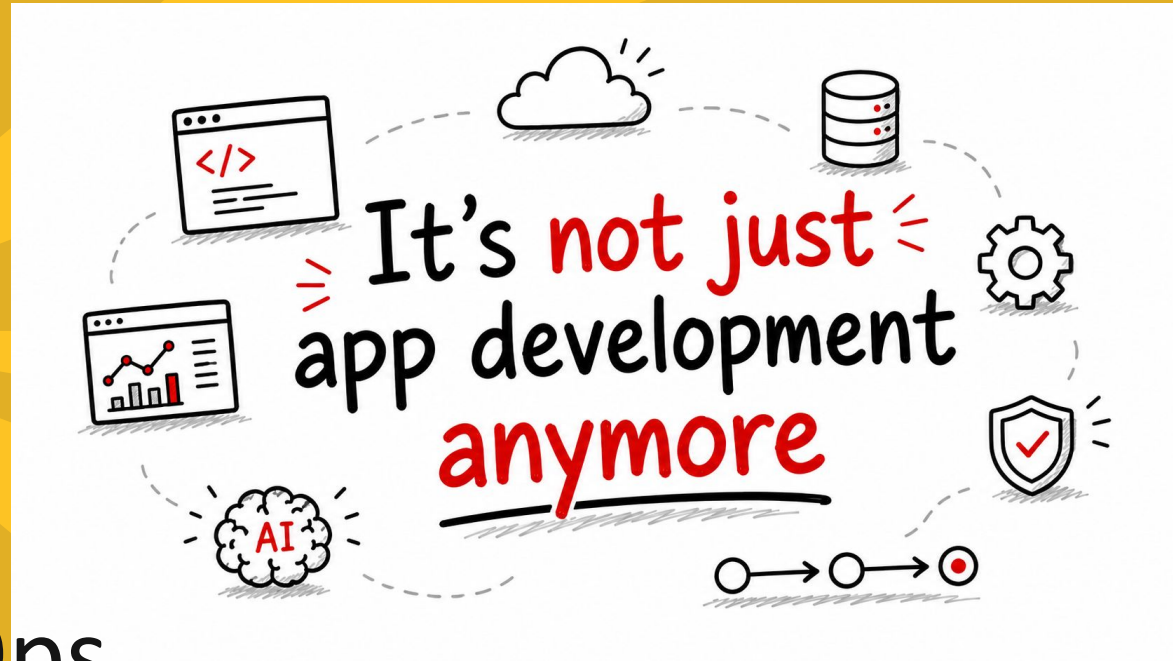
YOU RUN AWAY IT



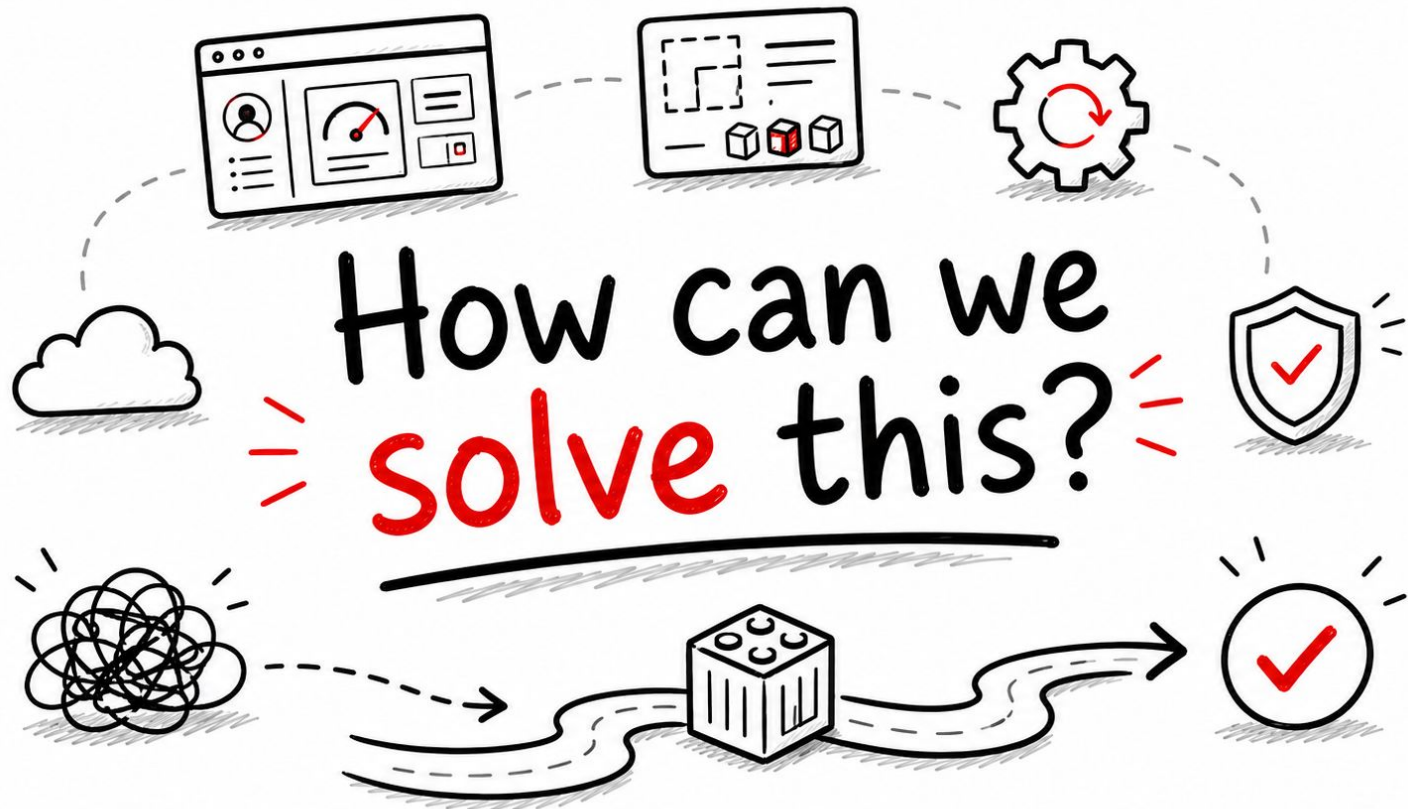
shift left,
increasing
cognitive load

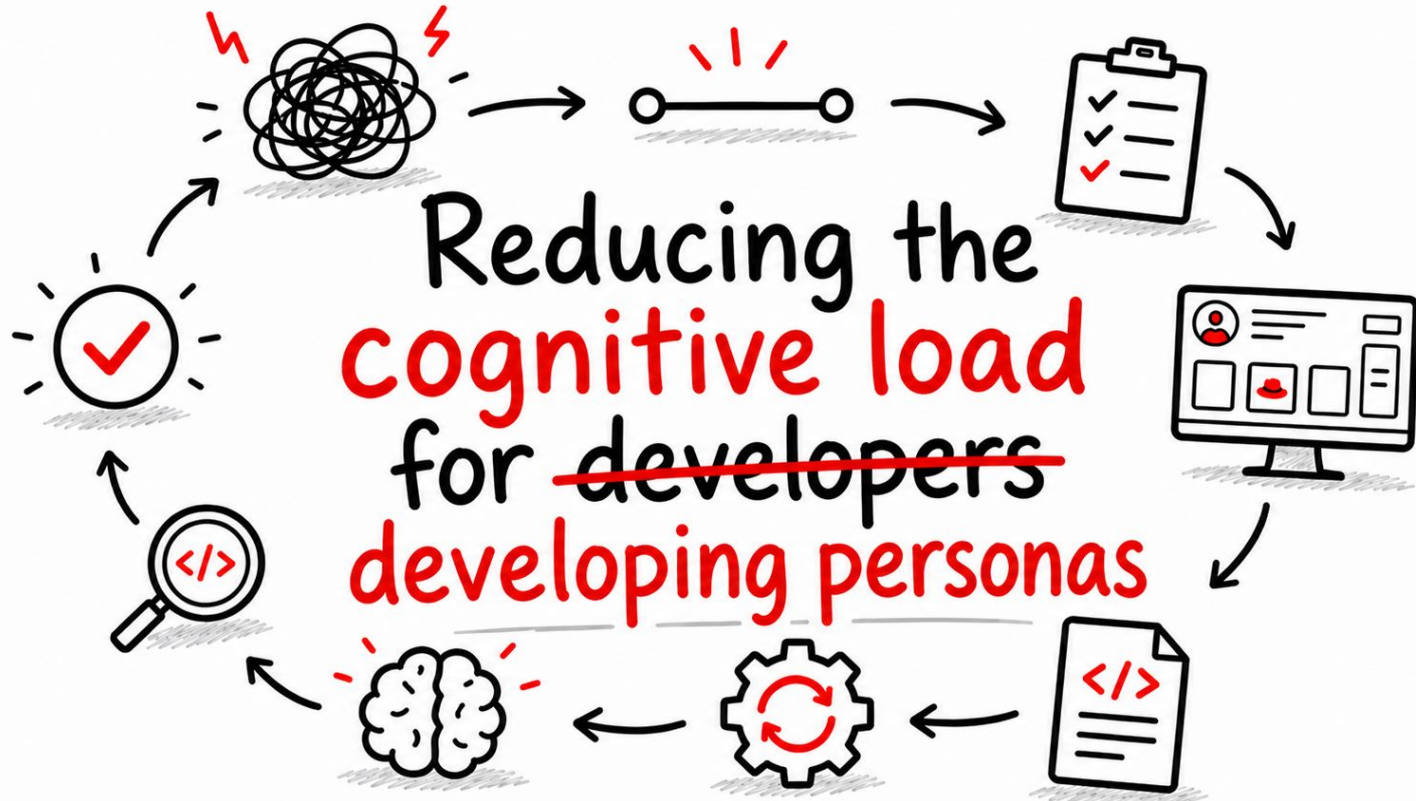


Dev
Ops
DevOps
FinOps
MLOps
GitOps
DataOps
DevSecOps
BizDevOps
BizDevSecOps
BizDevSecFinMLOps

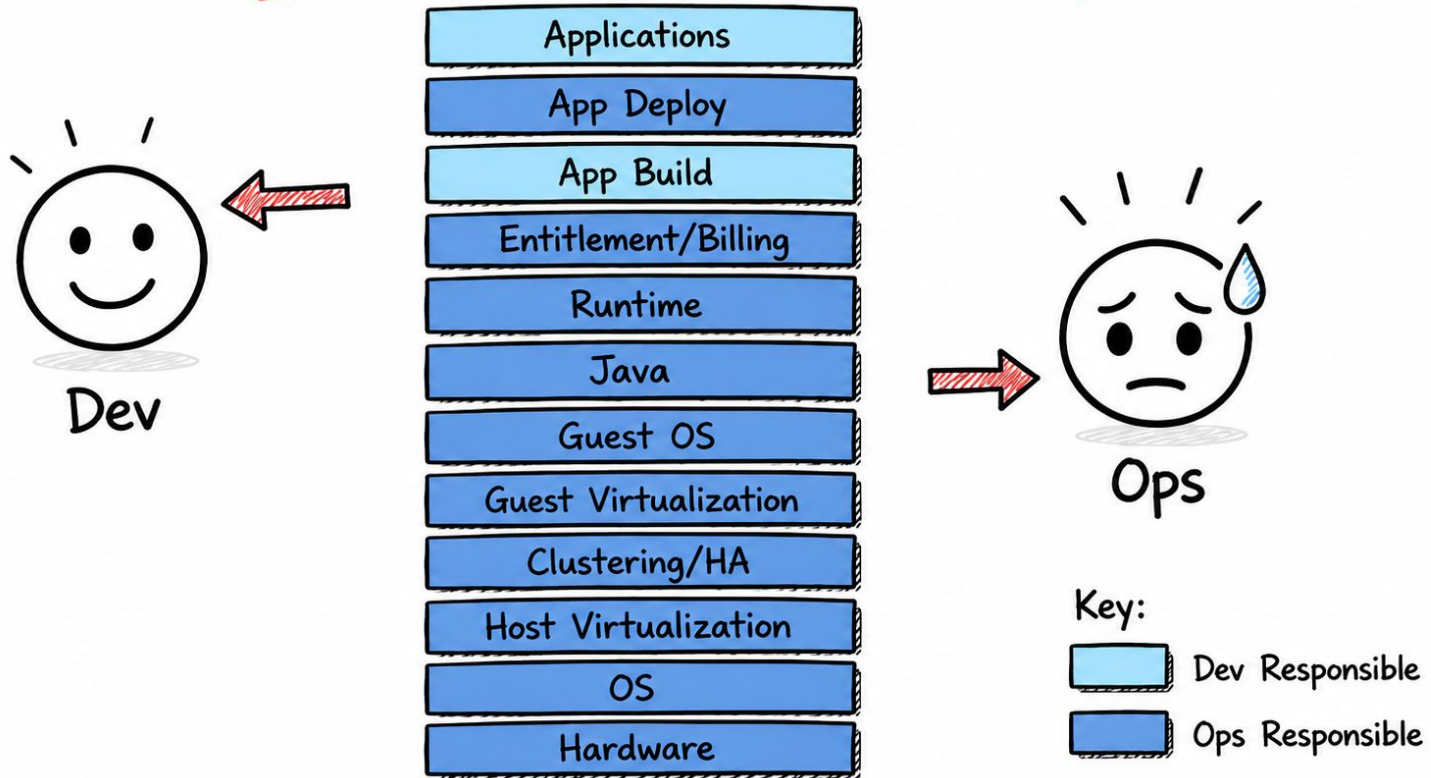








Traditional App Delivery



Platform engineering

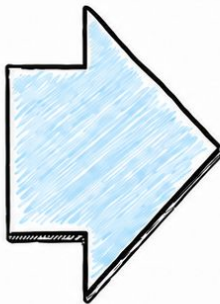
Traditional App Delivery



Dev



Ops



Key:



Dev Responsible



Ops Responsible

Modern App Delivery

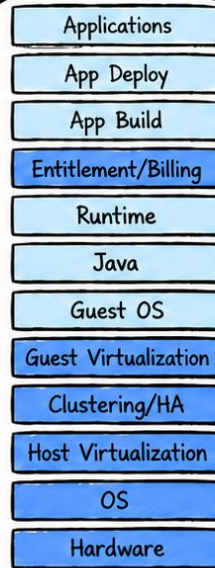
Usability

Observability

Quality

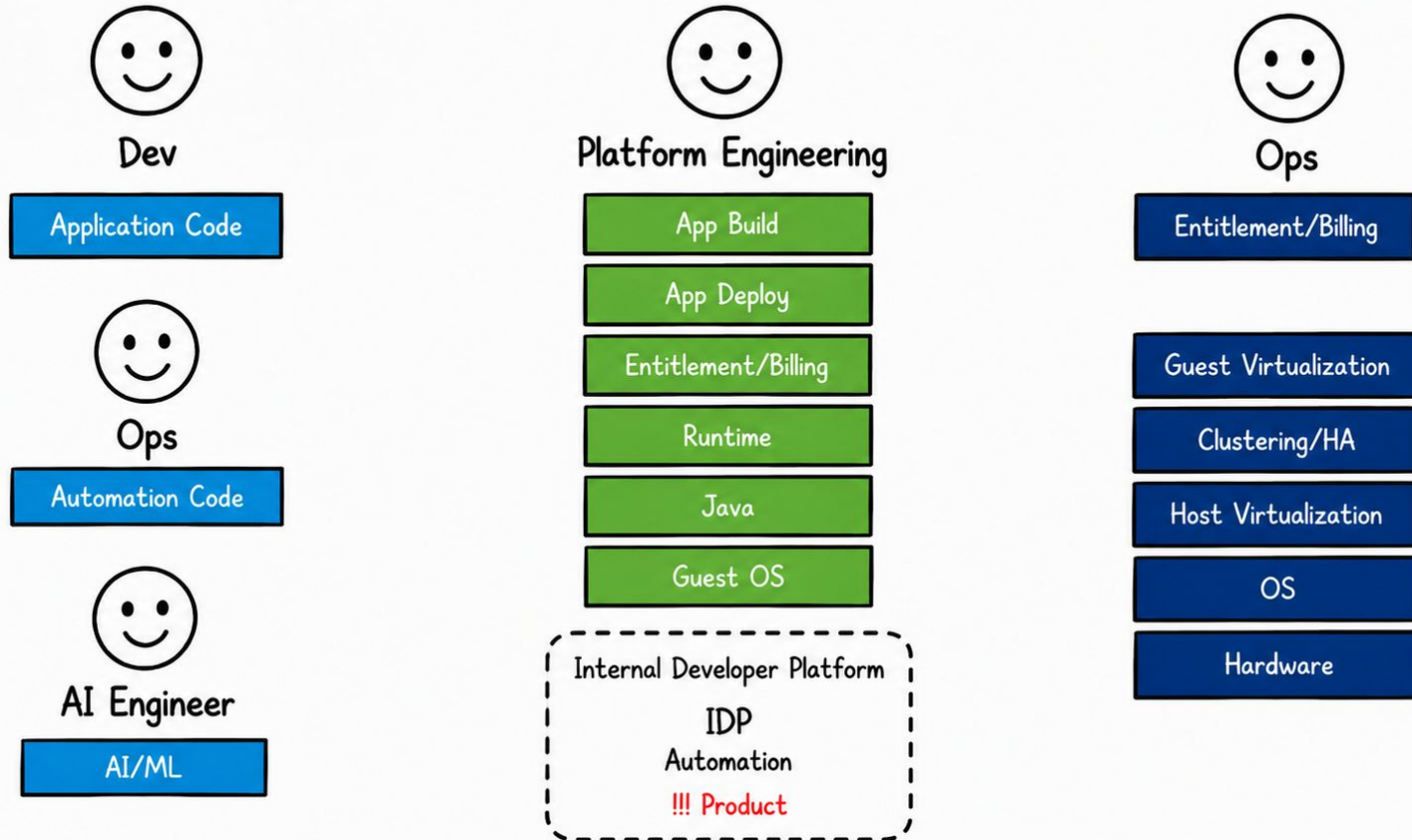


Dev

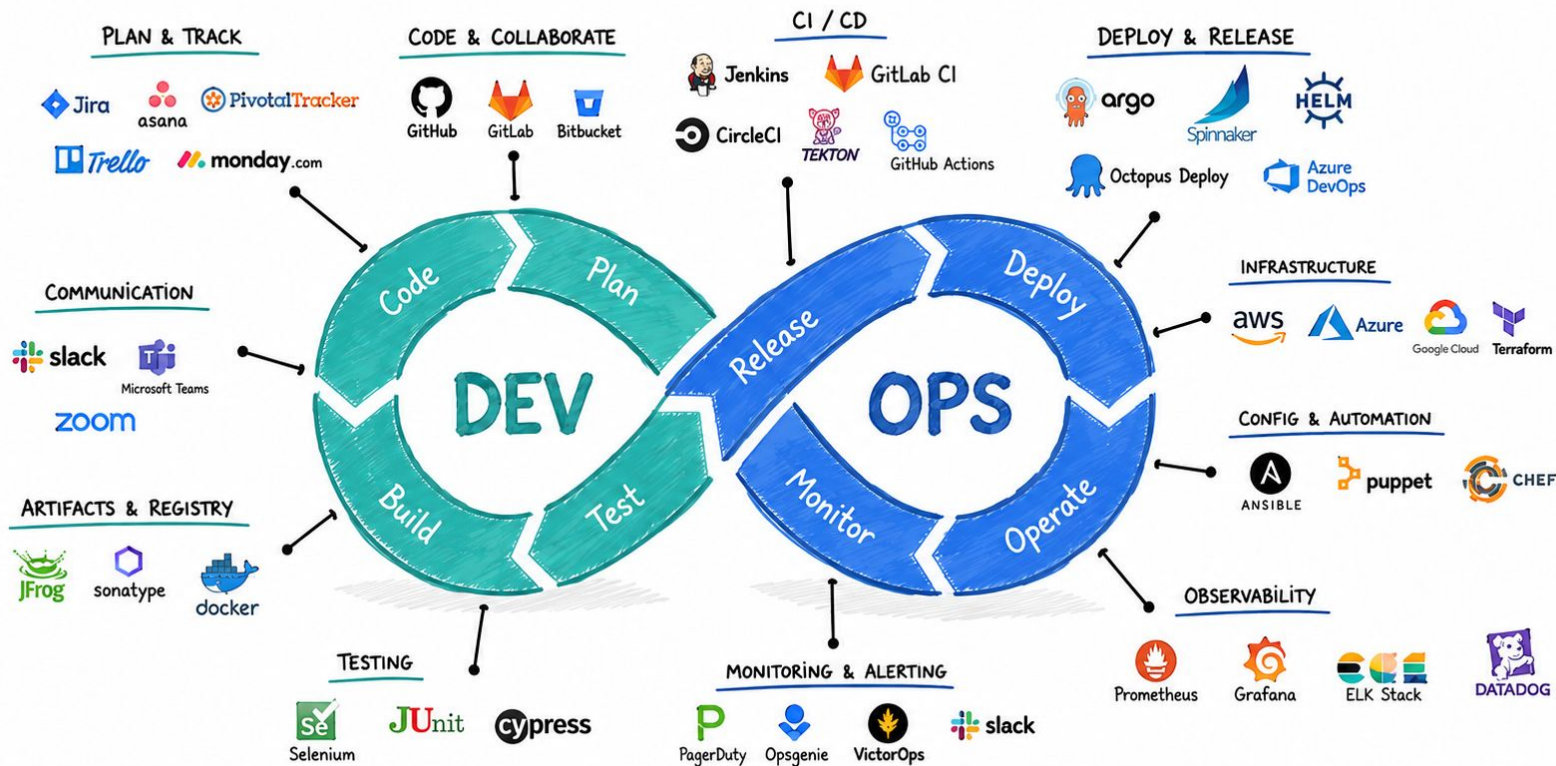


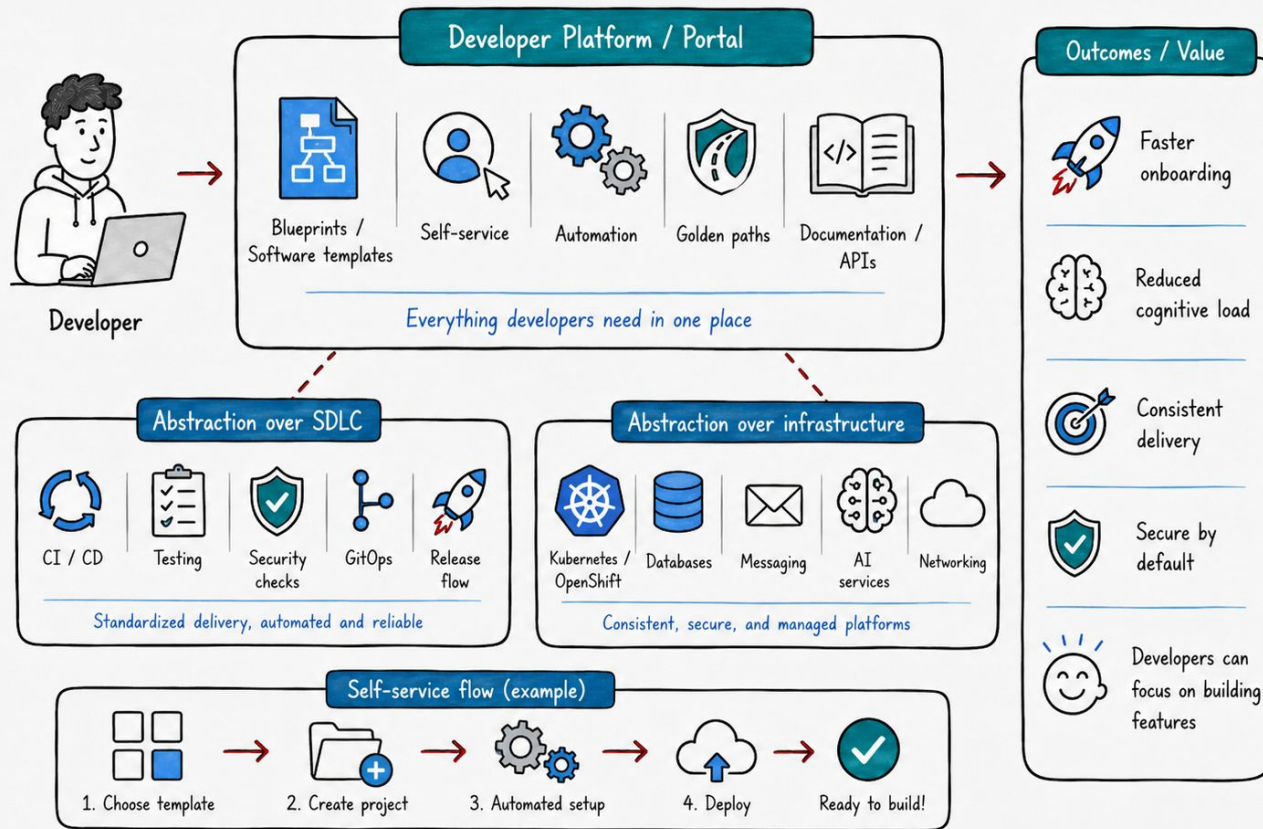
Ops

= Platform engineering =

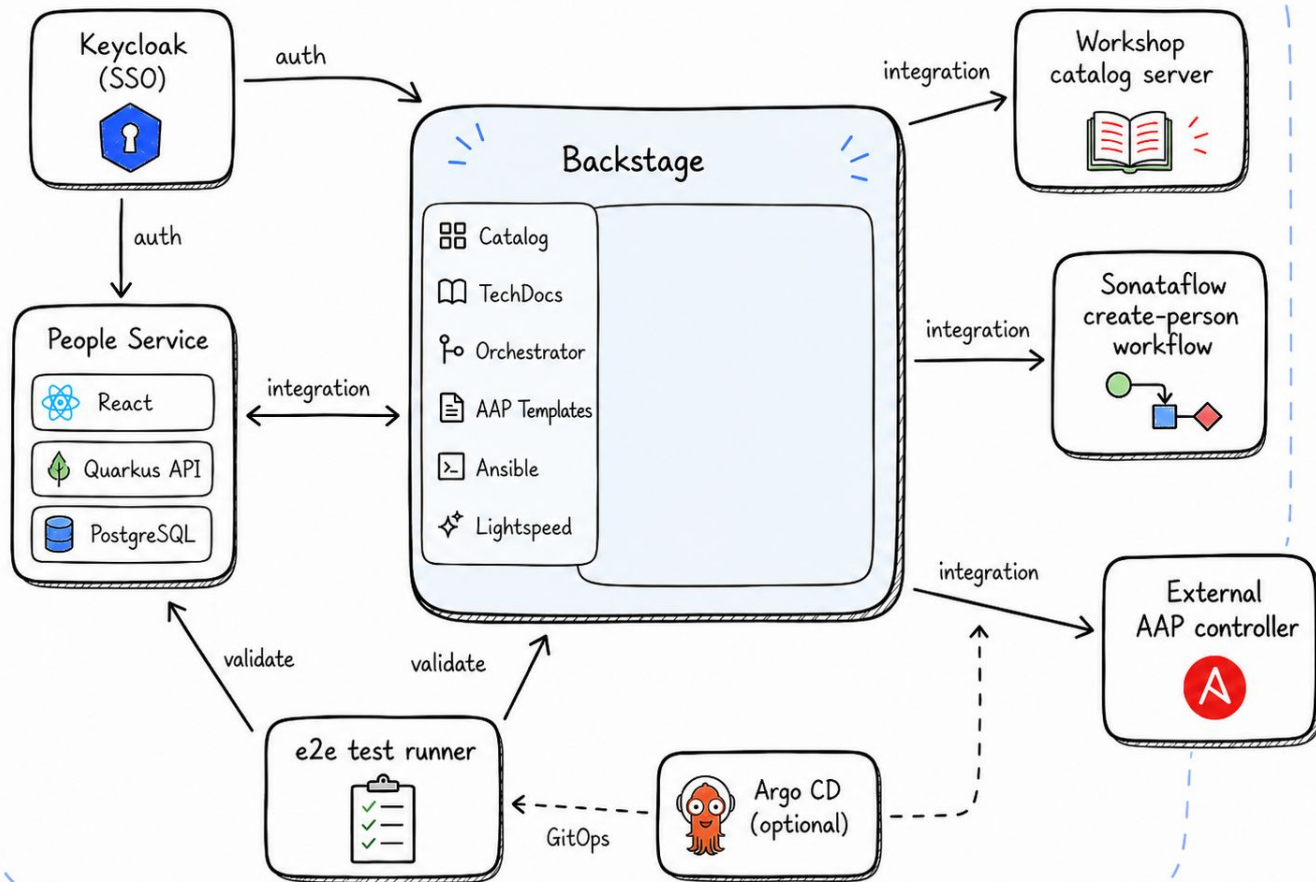


"Bake your own platform"





Kubernetes sandbox namespace



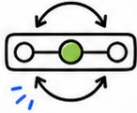
A man with a beard, wearing a red fedora and blue-tinted sunglasses, is smiling and looking upwards. He is wearing a black t-shirt with the Red Hat logo. In the background, a large white thought bubble with a black outline contains the text "Where do we start?". The background is red with yellow and orange diagonal streaks radiating from the center.

**Where do
we start?**

Developer Challenges

What is slowing developers down?

List the
main pain points



1

2

3

4

5

6

7

8



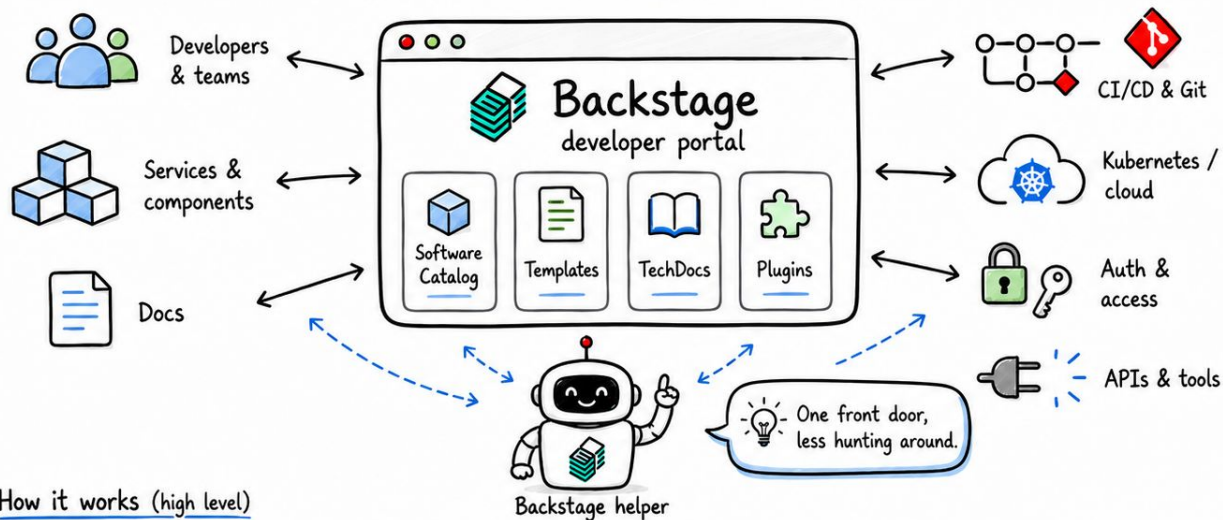
...

A man with a beard, wearing a red fedora-style hat and blue-tinted sunglasses, is smiling and looking upwards. He is wearing a black t-shirt with the Red Hat logo and the text "Red Hat" on the left chest. In the background, a large white thought bubble with a thick black outline contains the word "Backstage" in a bold, dark blue, sans-serif font. The entire scene is set against a vibrant red background with numerous yellow and orange diagonal lines radiating from the center, creating a dynamic, comic-book-like effect. The thought bubble has a small tail pointing towards the man's head, and there are three small white circles along the line connecting the bubble to his head.

Backstage

Backstage explained

An open platform for building your developer portal



How it works (high level)



What Backstage helps with



Catalog



Docs



Self-service



Integrations



Ownership



Standards

Lower cognitive load

Faster onboarding

Better discoverability

Less portal sprawl

Shared developer experience

Operators Vs Helm





Operators on OpenShift

Automate operations. Manage the moving parts.

- You build
- Operator manages
- OpenShift provides

What is an Operator?



An Operator is app-specific automation, packaged as software, that uses Kubernetes APIs to manage the app and its components.

Think of it as:



Domain expert
for your app

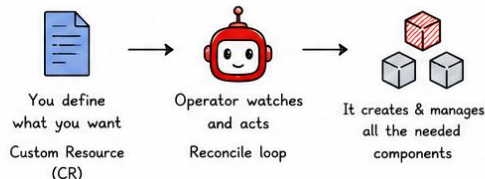


Encodes best
practices



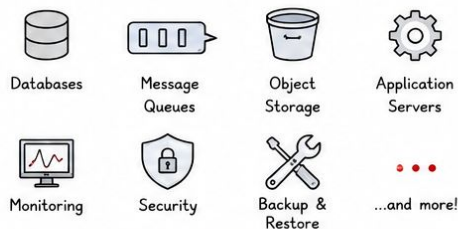
Ensures the app
stays healthy

How it works



Continuously compares desired vs. actual state and converges to the desired state

What an Operator manages



Anything stateful or complex that needs lifecycle management

Managing the moving parts

Operators handle the complexity so you can focus on your application.



Install & Upgrade

Automates installation, upgrades and version management.



Configuration

Manages configuration changes safely and consistently.



Health & Healing

Monitors components and automatically heals issues.



Scaling

Scales components based on demand or policies.



Backups

Schedules and manages backups and restores.



Security

Applies security best practices and keeps things up to date.



Lifecycle

Handles day-2 operations across the application lifecycle.

Where Operators run

Your Application



Operator

OpenShift (Kubernetes)



Operators run in-cluster and use the Kubernetes API to manage resources on your behalf.

Why it matters



Less manual
work



Faster, safer
delivery



More reliable
operations



Consistency across
teams & environments

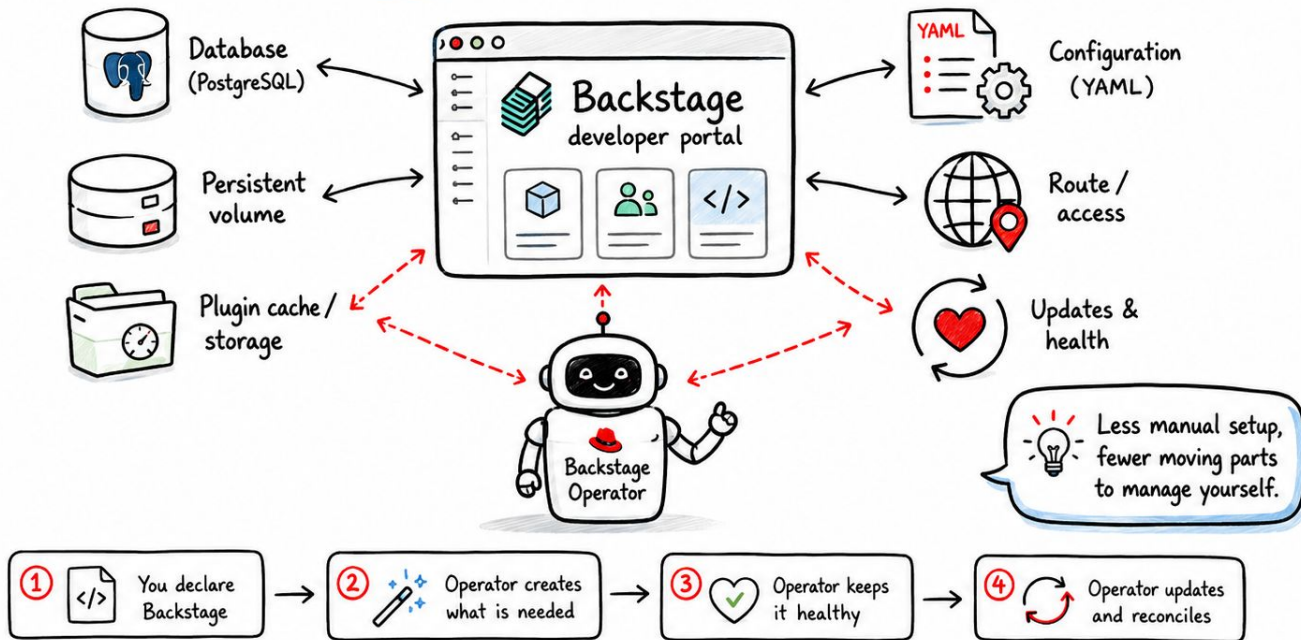


Lower risk and
operational cost

Operator example: Backstage on OpenShift



The Operator manages the moving parts for you



What the Operator handles



Install



Configure



Connect database



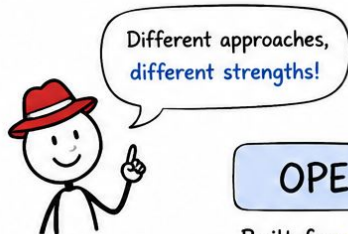
Manage storage



Recover / heal



Upgrade



OPERATOR vs HELM

Two ways to install software on OpenShift

OPERATOR



Built for **day-2 operations**



An Operator runs in the cluster and **continuously manages** your application



Continuous Reconciliation

Watches the cluster and corrects drift automatically



Lifecycle Management

Handles install, upgrades, rollbacks, and clean-up



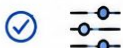
Resilience

Self-healing, handles failures and restarts



Managing Complex Apps

Understands the app's components and dependencies



Extensibility

Can add custom logic and integrations



Great for stateful, complex, and mission-critical apps that need **reliable, automated operations**

VS

HELM



Built for **packaging** and installation



Helm installs Kubernetes resources and **stops after deployment**



Package & Install

Bundles Kubernetes manifests into charts



Simple & Fast

Quick to deploy and easy to version



Configurable

Values files make customization easy



Great for Stateless & Simple Apps

Ideal for apps that don't need continuous management



Huge Ecosystem

Large community and many reusable charts



Great for straightforward, stateless applications and teams that want **simple packaging and install**

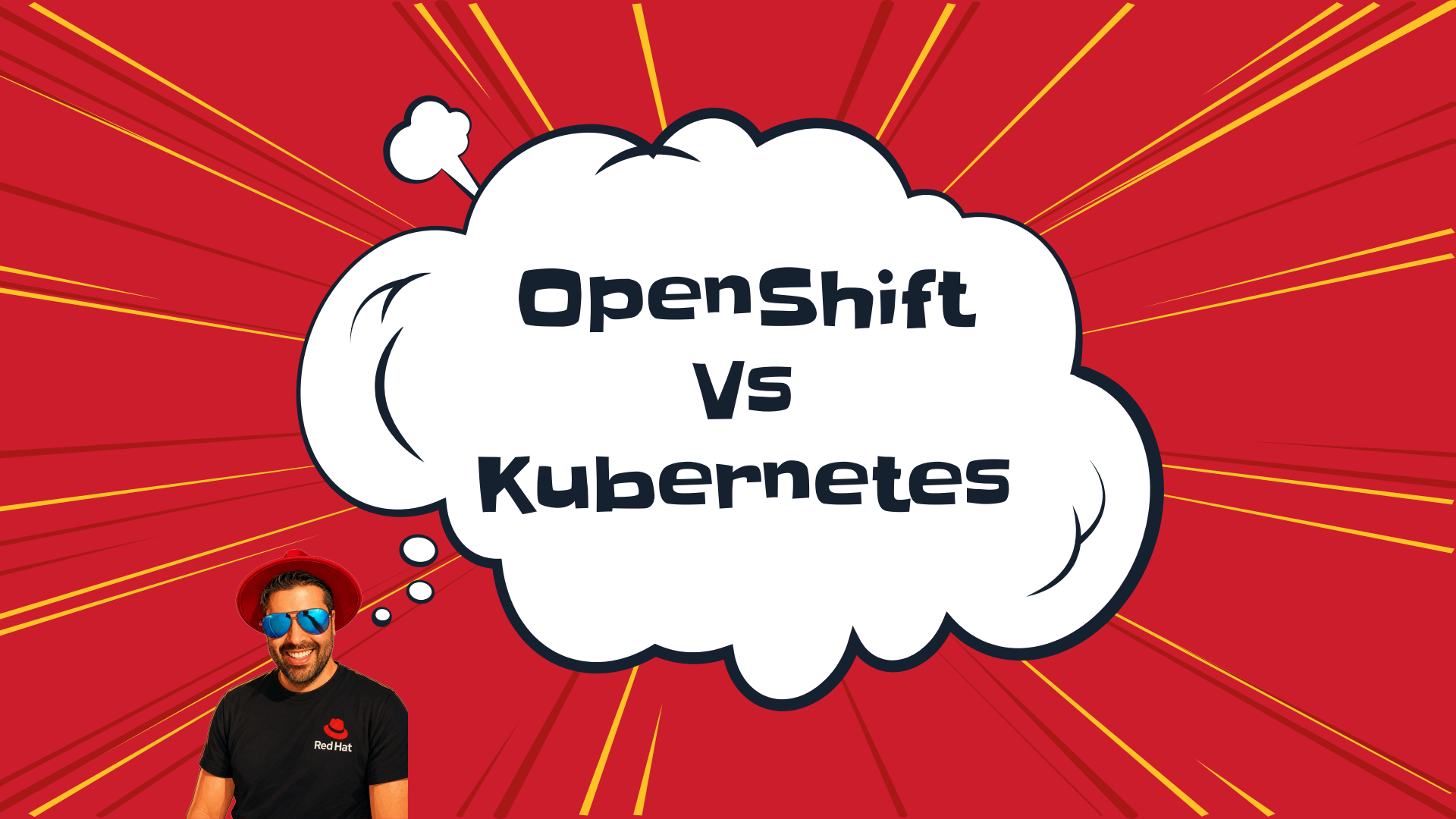


Choose the right tool:

Helm to get it in.

Operators to keep it running.



A man with a beard, wearing a red fedora-style hat and blue-tinted sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. He is positioned in the bottom left corner of the image. A large, white, cloud-like thought bubble with a black outline is centered in the image. Inside the bubble, the text 'OpenShift Vs Kubernetes' is written in a bold, dark blue font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from the center, creating a dynamic, comic-book-like effect. The thought bubble has a small tail pointing towards the man's head.

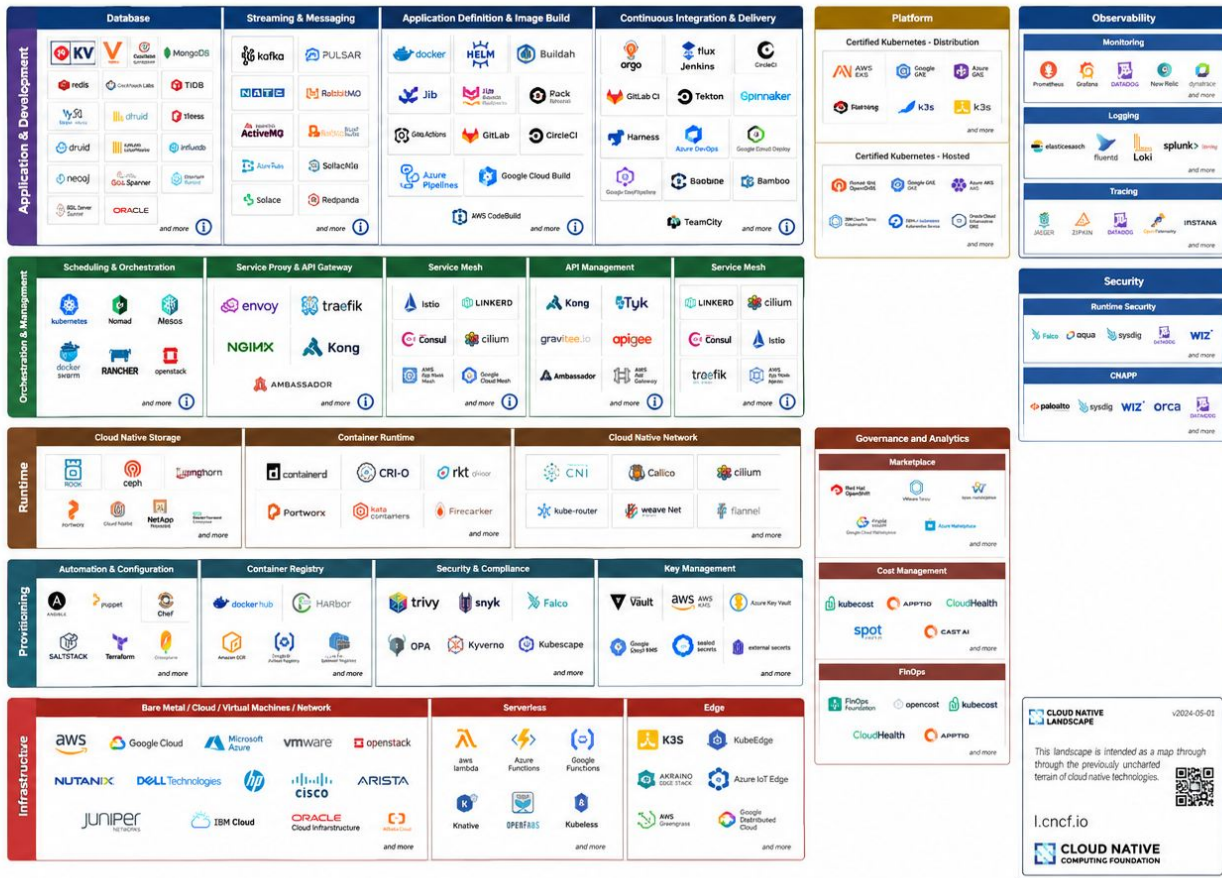
OpenShift Vs Kubernetes

The Cloud-Native Grocery Store

Where's Waldo?

or Wally?

or Charly?

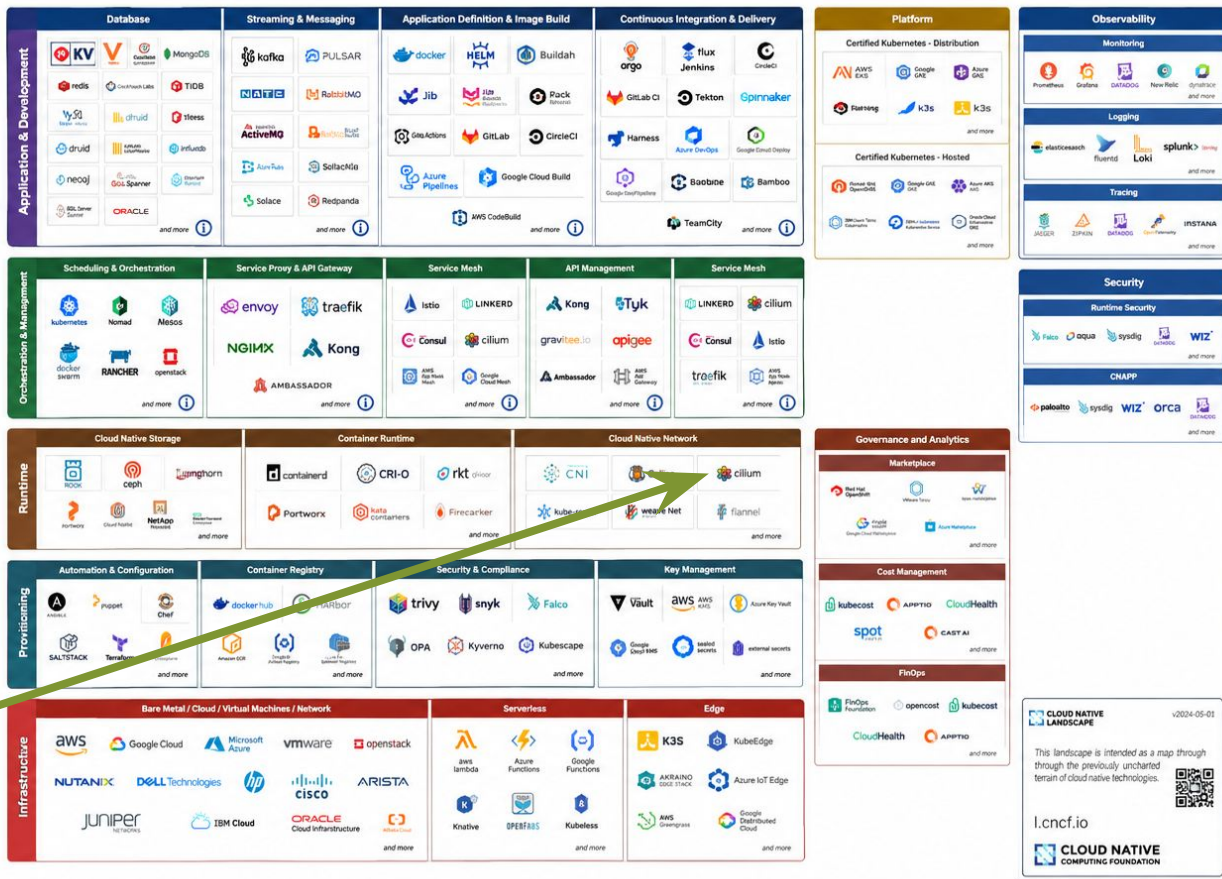


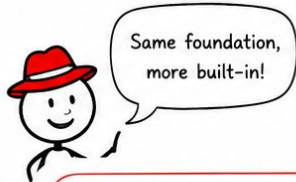
The Cloud-Native Grocery Store

Where's Waldo?

or Wally?

or Charly?





OpenShift vs Kubernetes

Kubernetes at the core. OpenShift adds enterprise superpowers.



OpenShift

Enterprise Kubernetes Platform



Enterprise Ready

Hardened, tested, and supported for production



Built-in Developer Experience

Web console, CLI, APIs, and IDE integration



Security by Default

RBAC, SCC, image scanning, network policies, and more



Integrated Platform Services

Logging, monitoring, registry, CI/CD, and service mesh



Enterprise Support

24/7 support, SLAs, and life cycle management



Faster Time to Value

Pre-integrated, pre-tested, and ready to run

VS



Kubernetes

The Container Orchestration Standard



Open & Extensible

Open source, community-driven, and highly extensible



DIY Approach

Build your platform with the tools you choose



Lightweight Core

Minimal core features, install what you need



Bring Your Own Tools

Choose and integrate your preferred solutions



Community Support

Large community, docs, and best practices



Maximum Flexibility

Full control to tailor to your environment



In short:

OpenShift = Kubernetes +
security, operations, and productivity

Kubernetes =
flexible foundation to build on



A man with a beard, wearing a red fedora and blue sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. To his right is a large, white, cloud-like speech bubble with a thick black outline. The bubble has a small tail pointing towards the man. Inside the bubble, the words "Bookmark hell" are written in a bold, black, sans-serif font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from behind the speech bubble, creating a sense of motion or impact.

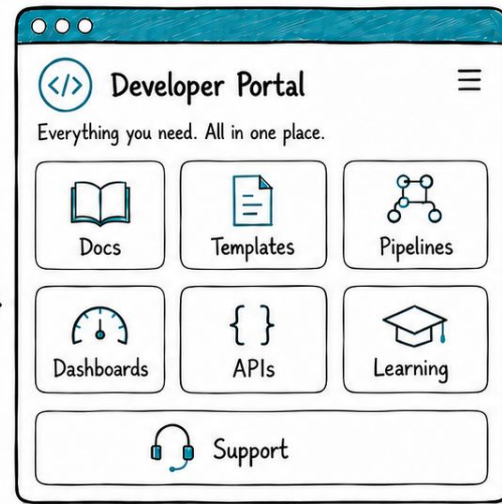
**Bookmark
hell**



Too many pages just to get work done.

DEVELOPER PORTAL

ONE ENTRY POINT



Find it fast

Trusted &
Governed

Built for
developers

FROM BOOKMARK CHAOS TO ONE DEVELOPER ENTRY POINT.

A man with a beard, wearing a red fedora and blue sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. He is positioned in the bottom left corner of the image. A large, white, cloud-like thought bubble with a black outline is centered in the image. Inside the bubble, the text "VM Vs Containers" is written in a bold, black, sans-serif font. The background is a solid red color with many thin, yellow, diagonal lines radiating from the center, creating a dynamic, comic-book-like effect.

VM Vs Containers



Run applications
in a consistent,
reliable way!

VMs vs Containers

Same goal, different approach



VM (Virtual Machine)



Container

Virtual Machines



Virtualize hardware. Each VM runs a full OS.

App 1

Bins / Libs

Guest OS



VM

App 2

Bins / Libs

Guest OS



VM

App 3

Bins / Libs

Guest OS



VM

Hypervisor

Host Operating System



Physical Server



Strong isolation



Heavier



Lower density

Containers



Share the host OS kernel. Lightweight and portable.

App 1

Bins / Libs



Container

App 2

Bins / Libs



Container

App 3

Bins / Libs



Container

Container Runtime (e.g., containerd)

Host Operating System (Shared Kernel)



Physical Server



Process isolation



Lightweight



Higher density

Benefits of Containers



Fast startup



Efficient
resource usage



Portable across
environments



Consistent
deployments



Lower
infrastructure costs



Run applications
in a consistent
reliable way!

VMs vs Containers

What changes in your day-to-day way of working?



VM (Virtual Machine)



Container

1 Packaging

VM



- Large images (GBs)
- OS included
- Slower to build and share

Container



- Smaller images (MBs)
- App + dependencies only
- Faster to build and share

2 Configuration

VM



- More manual configuration
- Differences per environment

Container



- Config through code (.env, ConfigMap)
- More consistent and repeatable

3 Local development

VM



- Heavier local environments
- Slower to start and change

Container



- Lightweight and fast to start
- Closer to production

4 Debugging

VM



- Often slower and more manual
- Tooling less standardized

Container



- Fast exec / attach
- Smaller, familiar toolset

5 Logging

VM



- Log files on the VM
- Central logging is harder

Container



- Logs to stdout / stderr
- Easier to centralize

6 State

VM



- State on disk in the VM
- VM-level backups
- Scaling is harder

Container



- State outside the container
- Volumes / DB / object storage
- Easier to scale

7 Networking

VM



- More complex network setup
- Fixed IPs, firewall rules

Container



- Service discovery
- DNS, services, ingress
- More dynamic and flexible

8 Security

VM



- OS-level isolation
- Larger attack surface
- OS patching needed

Container



- Process-level isolation
- Smaller attack surface
- Base image scanning

9 CI/CD

VM



- Slower builds
- Heavier test environments
- Deploy per VM

Container



- Faster builds and tests
- Ephemeral environments
- Faster, consistent deploys

10 Ownership

VM



- Ops manages infra & OS
- Dev depends on ops
- More waiting for changes

Container



- Teams own their app
- Self-service platform model
- Faster iteration



In short:

Containers make daily work lighter, faster, and more consistent.
You deliver value more often, with less friction.



Faster
delivery



Consistent
environments



Less
complexity



More autonomy
& ownership

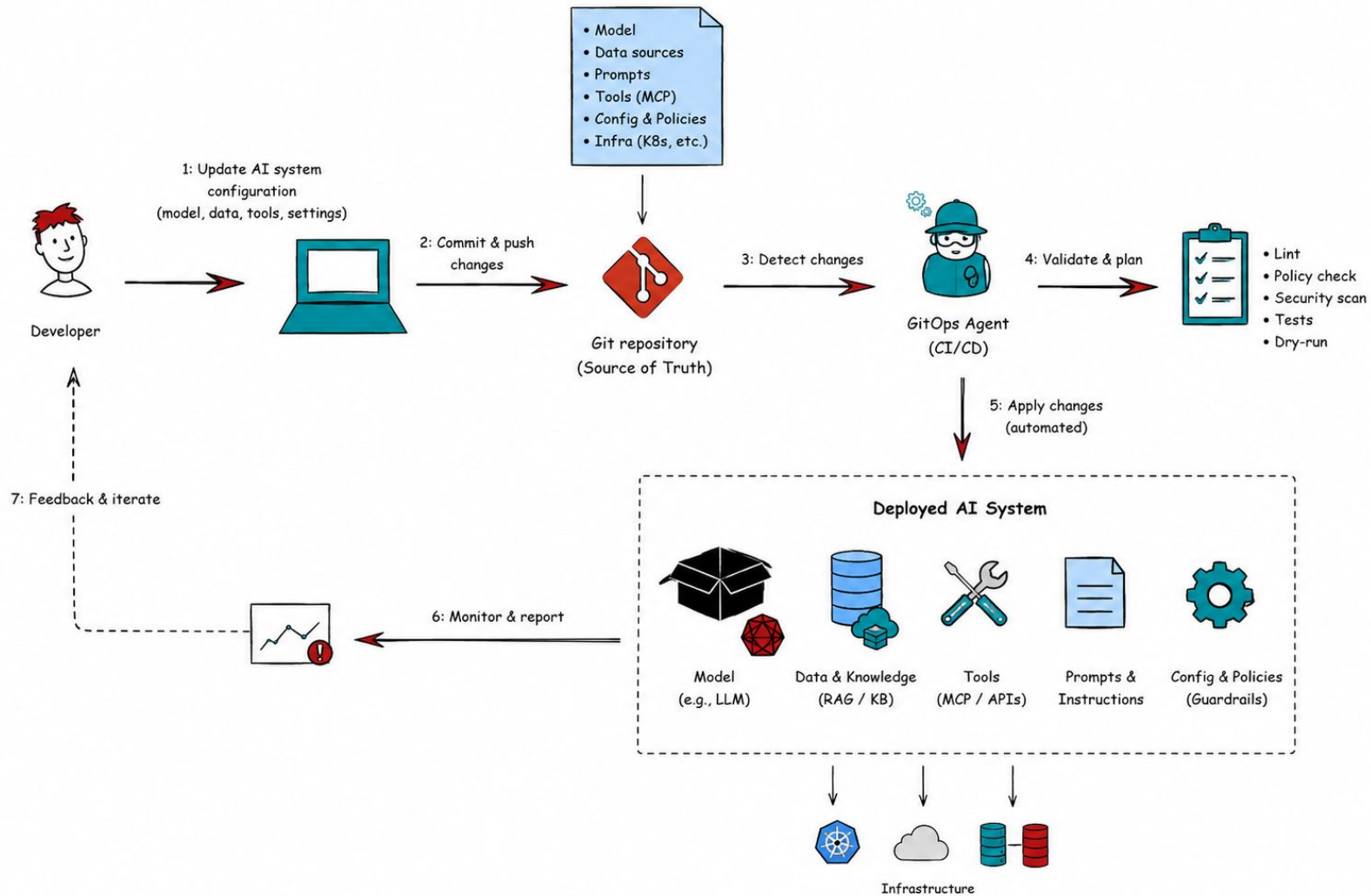


Safer by
default



GitOps


Red Hat

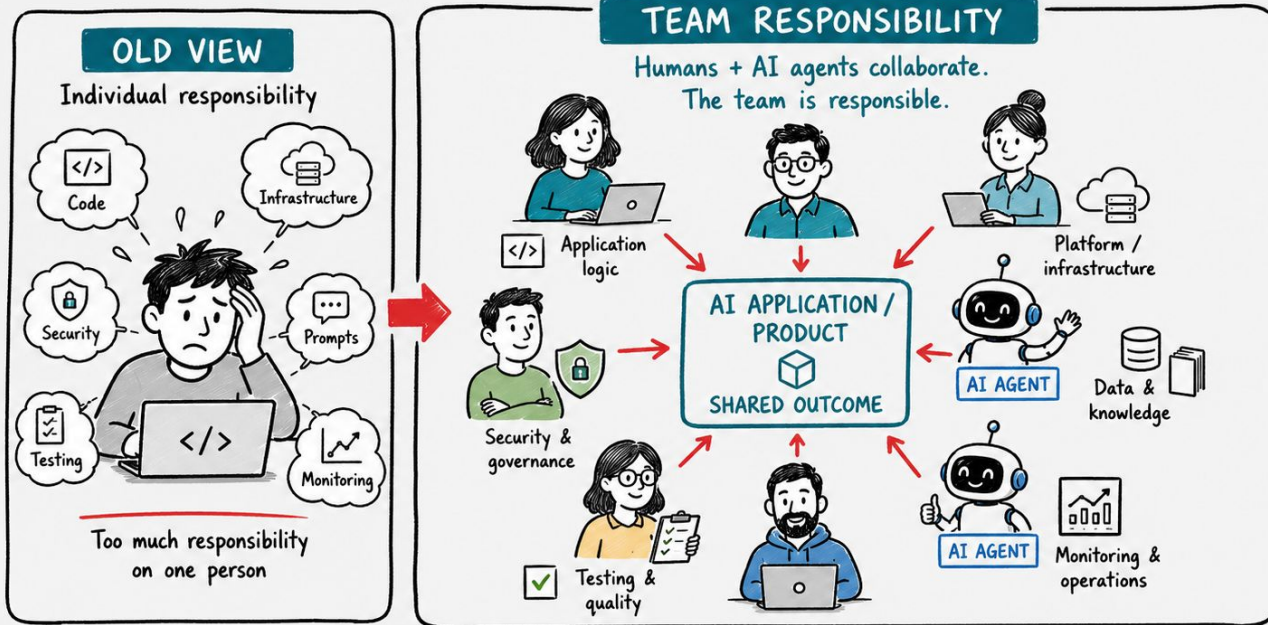




AI delivery is a team initiative



Responsibility shifts from one individual to a human-and-agent team.



THE RESULT:



Shared ownership



Reduced individual pressure

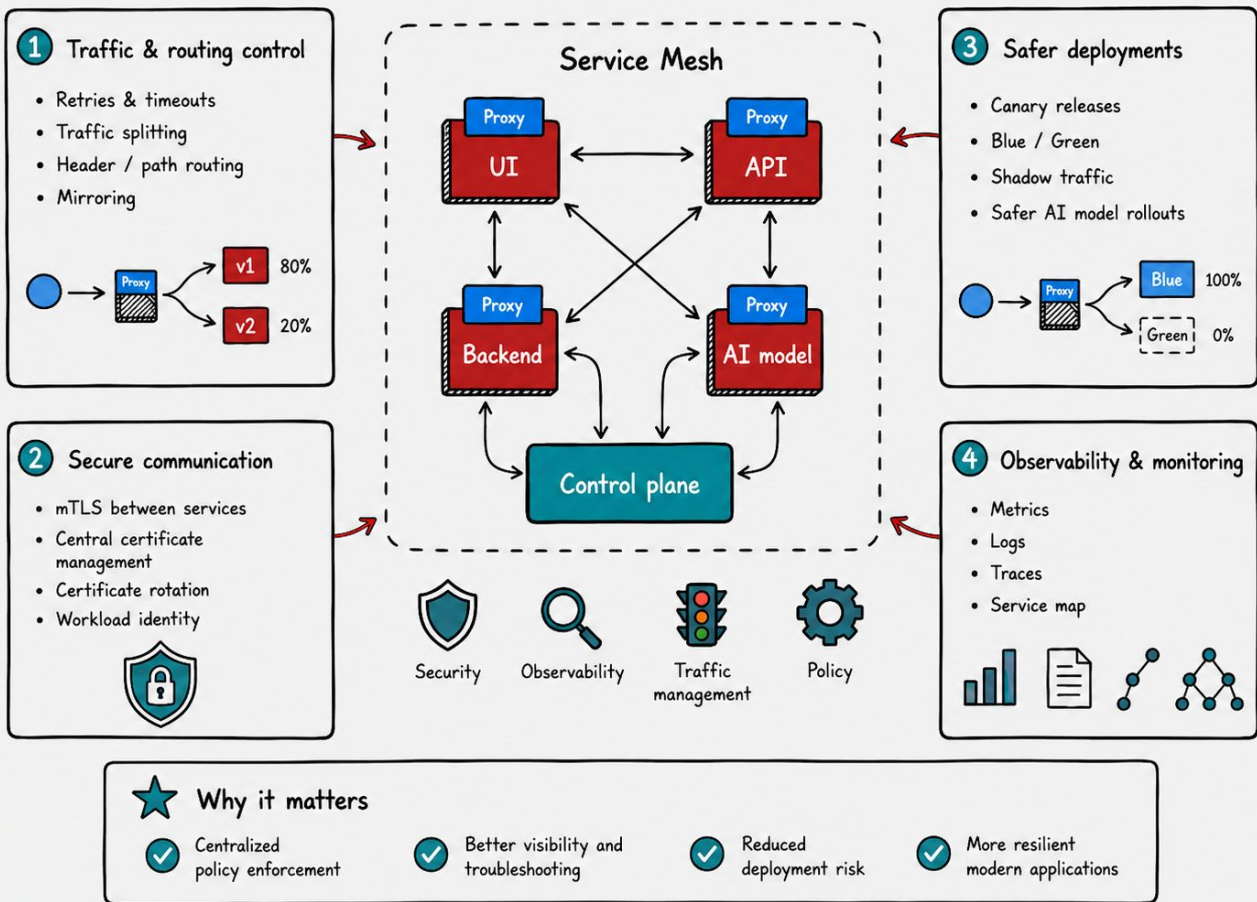


Better collaboration



More resilient delivery





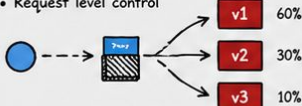
1 Centralized routing policies

- Retries
- Timeouts
- Circuit breaking
- Fault handling
- Policy consistency across services



2 Advanced routing policies

- Path or header based routing
- User or tenant specific routing
- Traffic splitting
- Mirroring
- Request level control



4 Centralized certificate management



- mTLS between services
- Certificate issuance and rotation
- Workload identity
- Encrypted service-to-service traffic

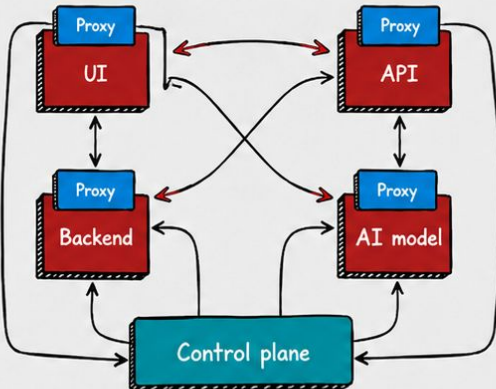


6 Why this matters for AI and modern apps

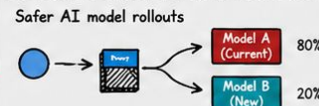


- ★ Consistent, secure communication between UI, API, backend, and AI model
- ★ Safer rollout of AI model and prompt changes (canary, mirroring)
- ★ Easier governance and policy enforcement across all services
- ★ Better reliability, performance, and visibility
- ★ Reduced operational risk in complex, distributed systems

Service Mesh



3 Advanced deployment patterns



5 Observability & monitoring



Resulting value

- ✓ More control over how traffic flows
- ✓ More security with identity & encryption
- ✓ Safer deployments with less risk
- ✓ Better visibility and faster troubleshooting
- ✓ Happier users and more resilient apps





Demo Platform

Argo Rollouts Workshop

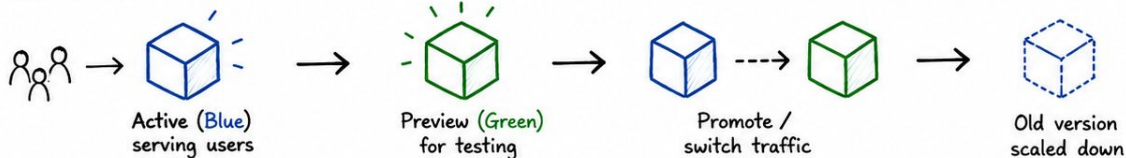
BlueGreen Rollout



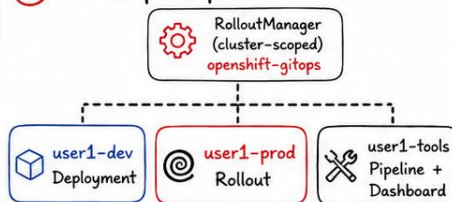
In this module we will:

- 1 Understand BlueGreen
- 2 Deploy rollout to Prod
- 3 Promote a new image
- 4 Roll back if needed

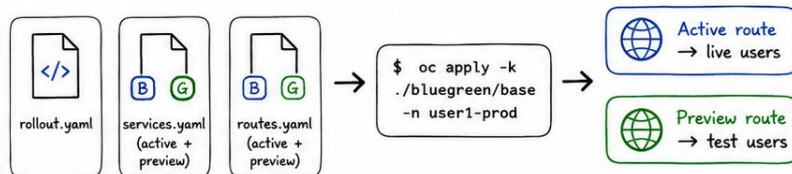
1 How BlueGreen works



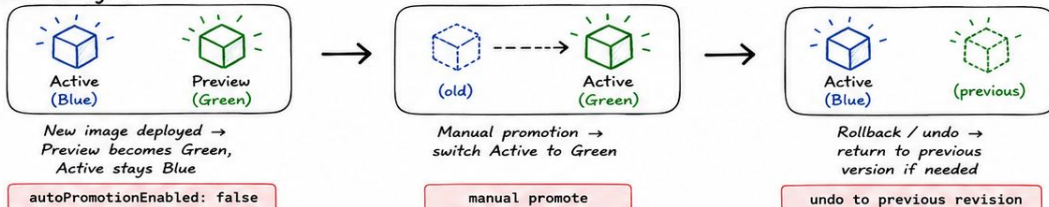
2 Workshop setup



3 Deploy the BlueGreen rollout



4 Promotion lifecycle



Blue = current active version



Green = new preview version



Argo Rollouts manages the switch





Demo Platform

Argo Rollouts Workshop

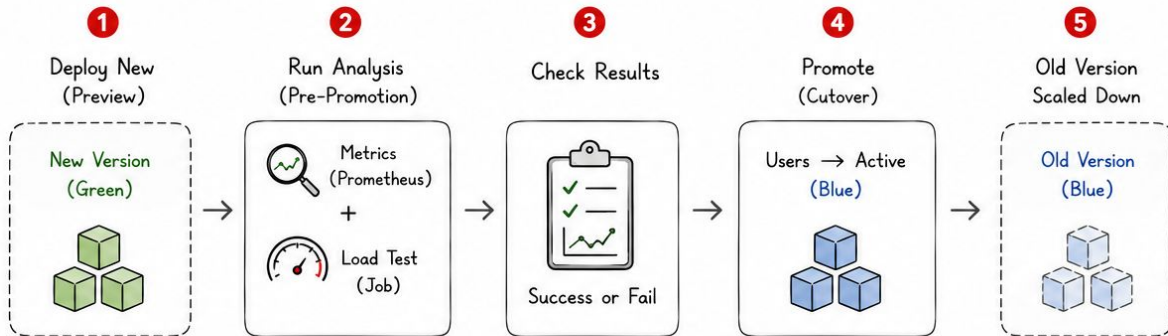
Analysis

In this module we will:

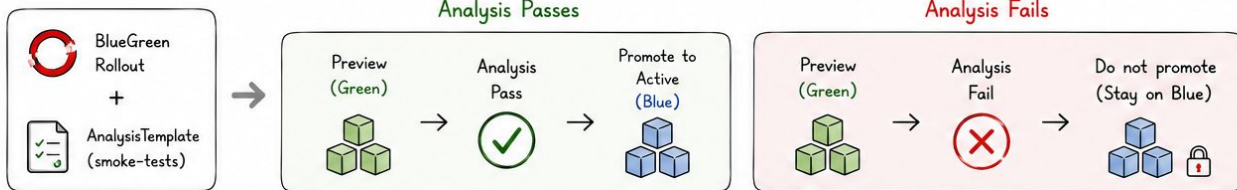
- ✓ Understand the concept of Analysis
- ✓ Deploy BlueGreen Rollout with Analysis
- ✓ Promote with Analysis
- ✓ Observe pass and fail scenarios



How Analysis
works (BlueGreen)



What we deploy



Application Version



New (Preview) Version



Active Version



Metrics (Prometheus)



Load Test (Job)



Argo Rollouts





Roll out changes
with confidence!

Demo Platform Argo Rollouts Workshop

Canary

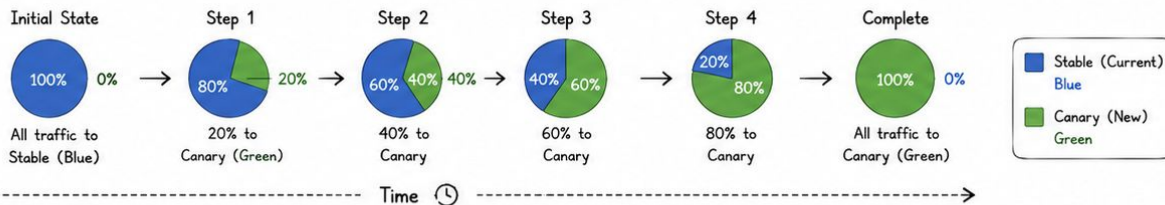
In this module we will:

- ✓ Understand the Canary Strategy
- ✓ Deploy Canary Rollout to Prod
- ✓ Promote a new image & observe
- ✓ Use inline analysis in the rollout

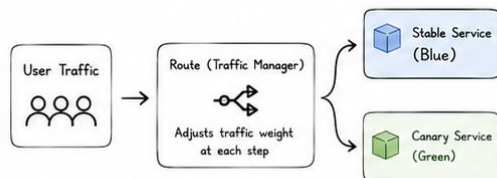


1 Canary Strategy - How it works

Traffic is shifted gradually from the current (stable) version to the new (canary) version in steps.



2 Canary Rollout Architecture



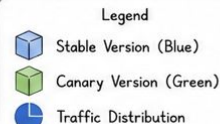
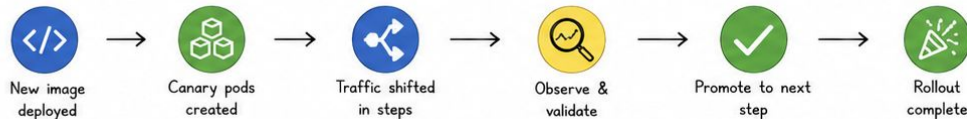
3 Canary Rollout Steps (Example)

Step	Action	Traffic to Canary	Pause
1	Set Weight	20%	Manual
2	Set Weight	40%	10s
3	Set Weight	60%	10s
4	Set Weight	80%	10s
5	Complete	100%	-

4 Key Components

	Rollout	Defines the canary strategy and rollout steps
	Services	Stable and Canary services receive traffic
	Routes	OpenShift Routes manage traffic weights
	OpenShift Traffic Manager	Automatically adjusts traffic between services

5 Canary Rollout Flow

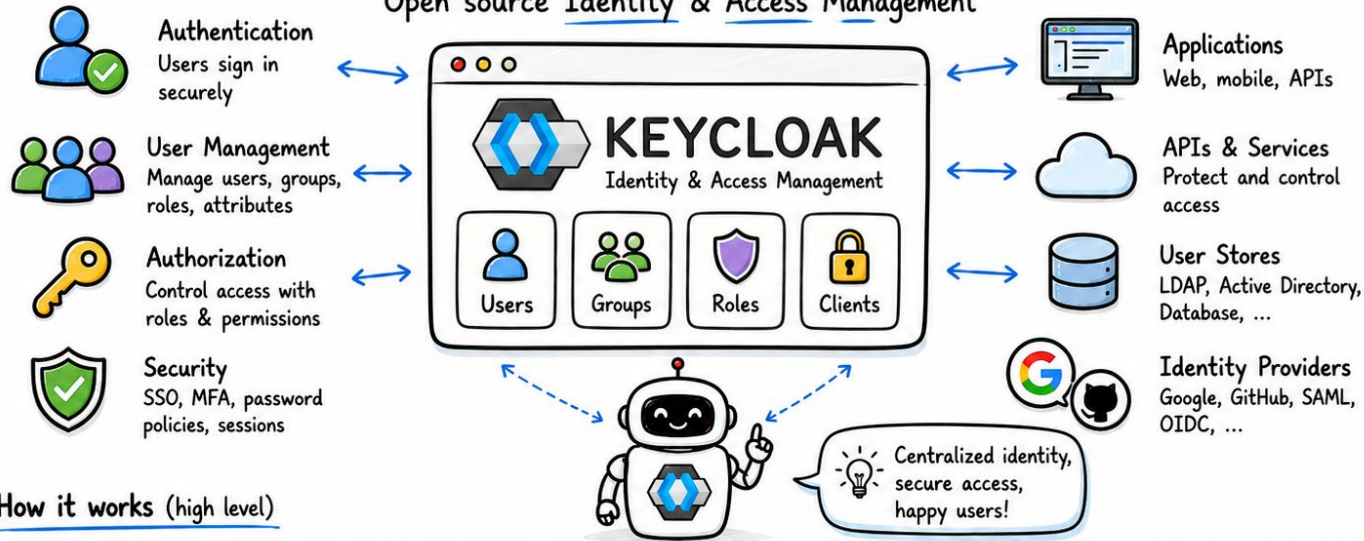


A man with a beard, wearing a red fedora and blue sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. To his right is a large, white, cloud-like thought bubble with a thick black outline. Inside the bubble, the word 'Keycloak' is written in a bold, dark blue, sans-serif font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from behind the thought bubble, creating a dynamic, comic-book-style effect. A small, white, cloud-like shape is positioned above the main bubble, and a series of three small white circles lead from the man's head towards the bottom left of the main bubble.

Keycloak

Keycloak explained

Open source Identity & Access Management



How it works (high level)



What Keycloak handles



A man with a beard, wearing a red fedora-style hat, blue sunglasses, and a black t-shirt with the Red Hat logo, is smiling. He is positioned in the bottom left corner of the image. A large, white, cloud-like thought bubble with a thick black outline is centered in the image. It has a small tail pointing towards the man's head. The bubble contains the text "SpringBoot Vs Quarkus" in a bold, dark blue font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from behind the thought bubble, creating a sense of energy and motion.

SpringBoot Vs Quarkus

Spring Boot vs Quarkus

Two leading Java frameworks for building modern applications



Spring Boot

Mature, opinionated, feature-rich



Wide range of features
Great for monoliths
and traditional apps



Startup time: Moderate
(memory: higher)



Large ecosystem
Many starters, libraries
and integrations



Developed for the JVM
Runs anywhere Java runs



Configuration: Flexible
application.properties / yaml



Large community
Many resources, tutorials
and enterprises



Focus



Performance



Ecosystem



Runtime



Configuration



Community



Quarkus

Cloud-native, fast, Kubernetes-first



Built for cloud-native
Microservices, serverless,
event-driven



Startup time: Very fast
(memory: low)



Growing ecosystem
Focused on cloud-native
technologies



Optimized for containers
Great for Kubernetes
and OpenShift



Configuration: Build-time
Leverages build-time optimization



Smaller but rapidly growing
Strong in cloud-native
and developer experience



Both are great!
Choose based on your
needs, team skills and
architecture.

Choose Spring Boot if you need:

- A mature, proven framework
- A wide range of features and libraries
- Monolithic or traditional applications
- Maximum flexibility and customization
- Large community and ecosystem



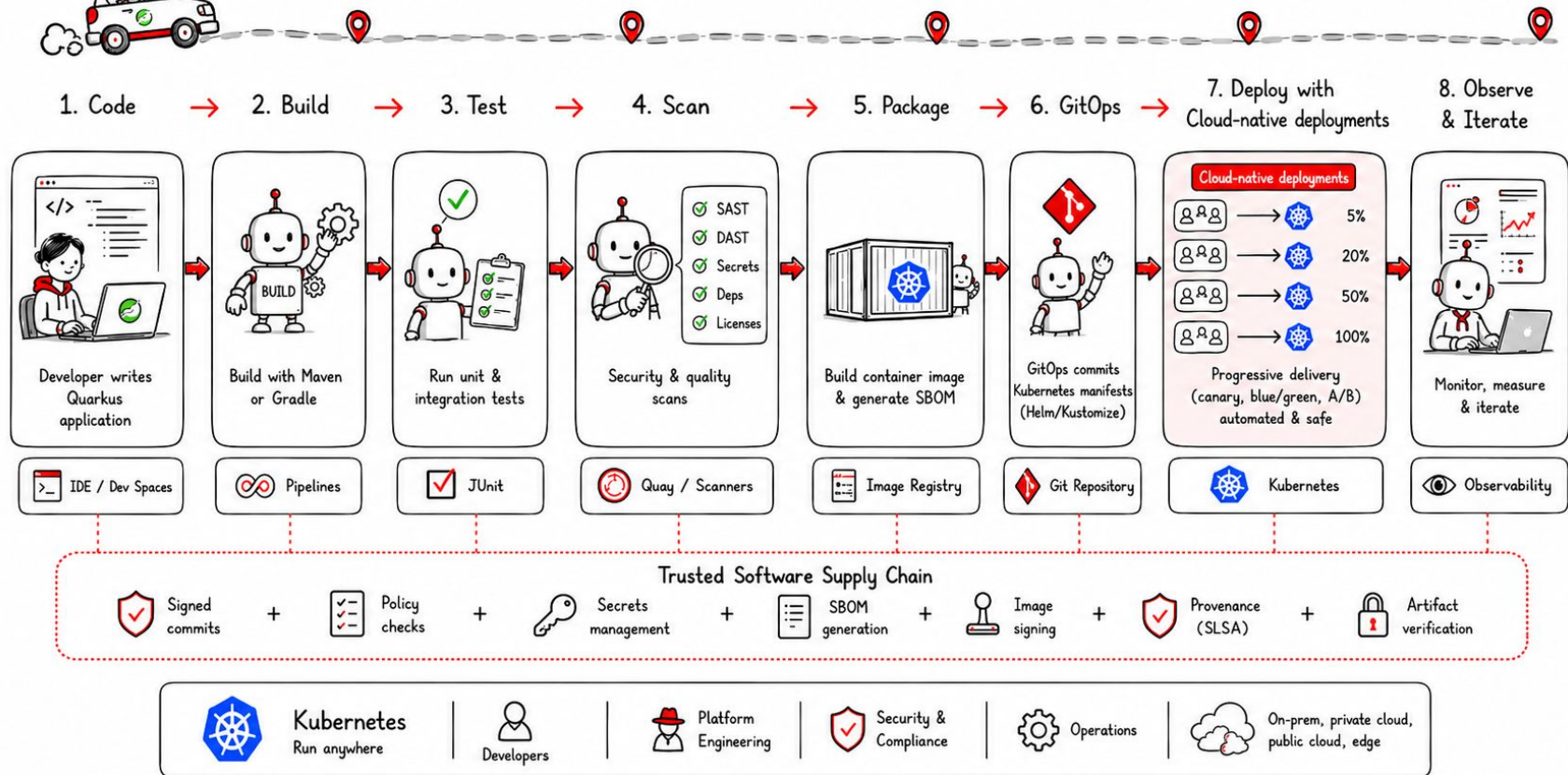
Choose Quarkus if you need:

- Fast startup and low memory
- Cloud-native / Kubernetes-first apps
- Microservices, serverless, events
- Build-time optimization
- Modern developer experience

A man with a beard, wearing a red fedora-style hat and blue-tinted sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. He is positioned in the bottom left corner of the image. A large, white, cloud-like speech bubble with a thick black outline is centered in the image. Inside the bubble, the text "Trusted Software Supply Chain" is written in a bold, dark blue, sans-serif font. The background is a vibrant red with numerous yellow and orange diagonal lines radiating from the center, creating a dynamic, comic-book-like effect. The speech bubble has a small tail pointing towards the man's head.

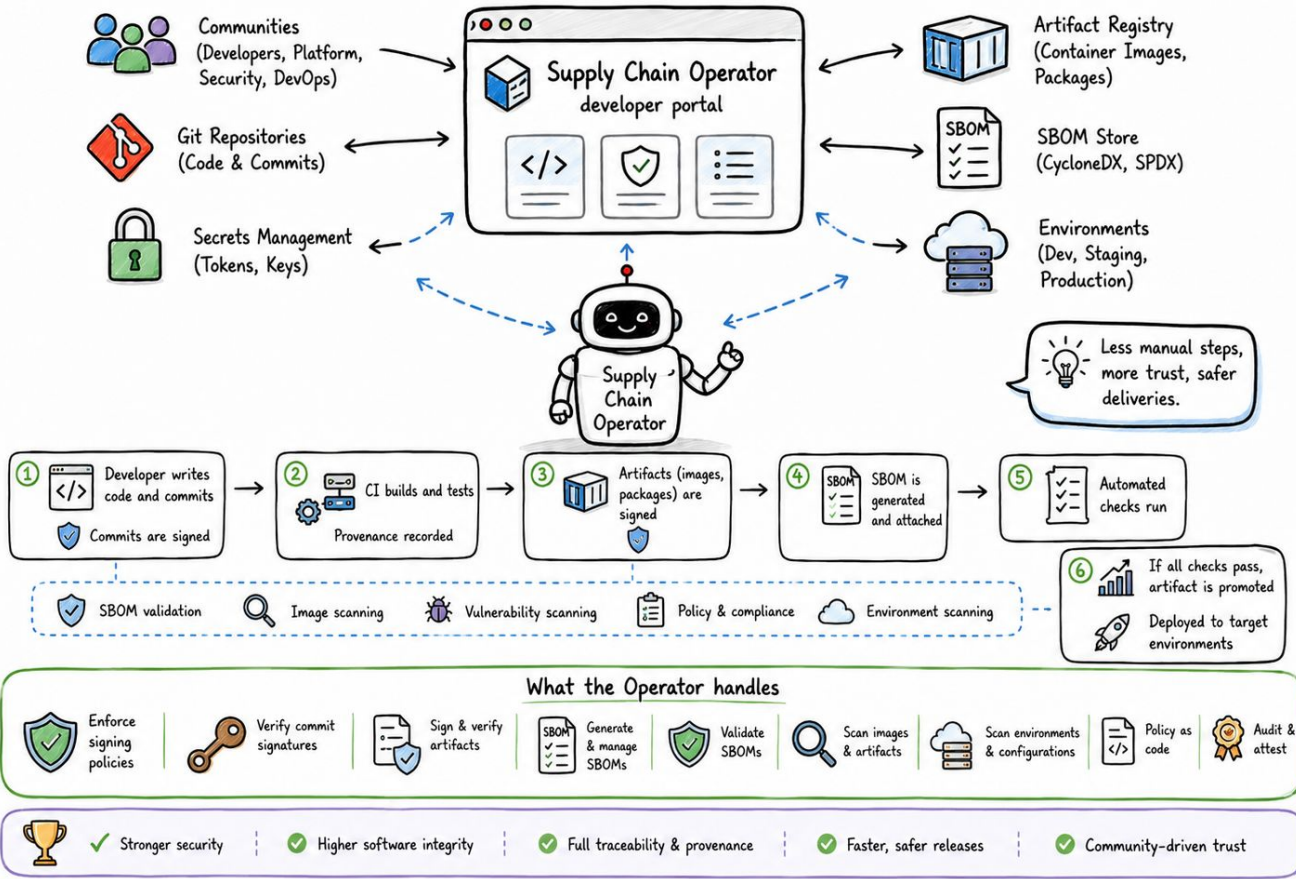
Trusted Software Supply Chain

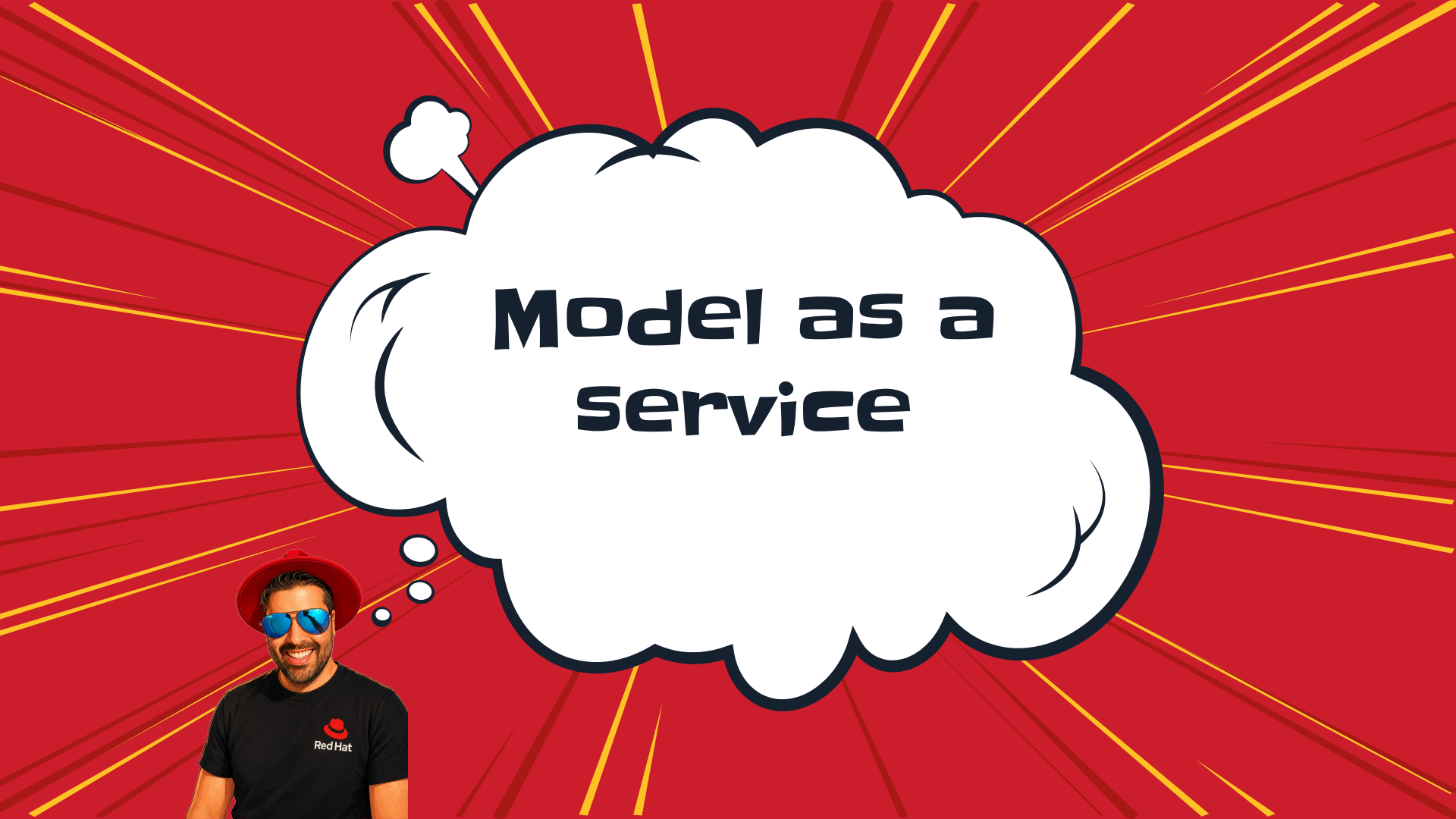
The road to deploy a ~~Spring Boot~~ Quarkus application on Kubernetes



Operator example: Trusted Software Supply Chain

The Operator automates and enforces trust across your software supply chain

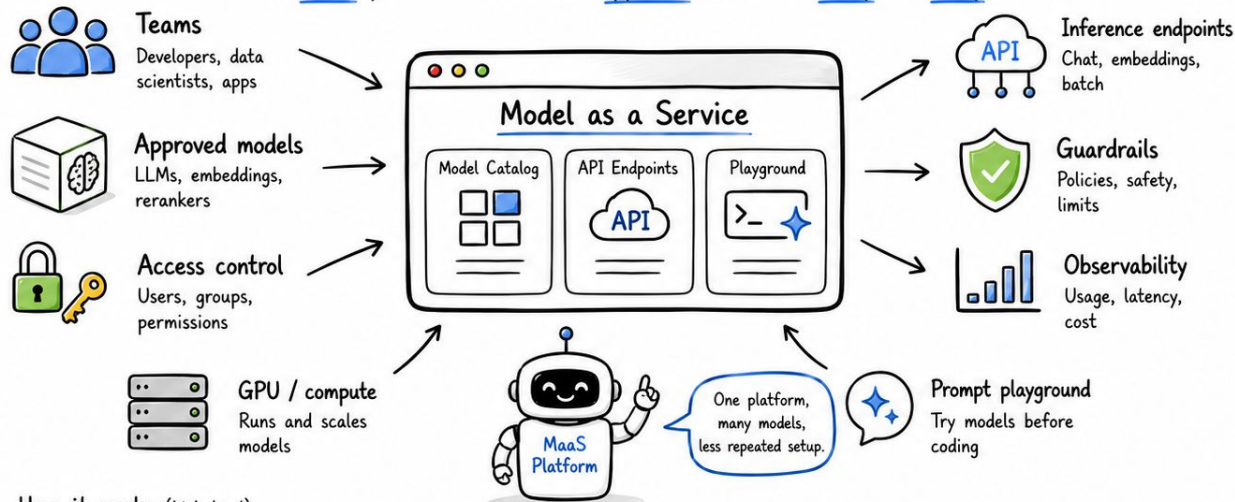


A man with a beard, wearing a red fedora and blue sunglasses, is smiling. He is wearing a black t-shirt with the Red Hat logo. To his right is a large, white, cloud-like thought bubble with a thick black outline. Inside the bubble, the text "Model as a service" is written in a bold, dark blue, sans-serif font. The background is a vibrant red with numerous yellow and orange diagonal streaks radiating from behind the thought bubble, creating a sense of energy and focus.

**Model as a
service**

Model as a Service explained

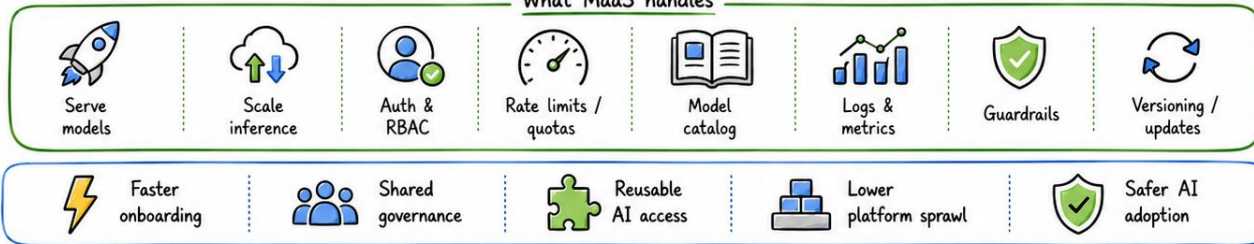
A shared platform to consume approved AI models safely and easily



How it works (high level)



What MaaS handles





What's next?



Reduce your cognitive load

“Get started!”

maarten-vandeperre.be
developers.redhat.com

